



A security risk assessment approach based on improved PSO-BPNN for intelligent network information systems

Tong Ye¹ · Gongzhe Qiao²

Received: 17 April 2024 / Accepted: 2 September 2026
© The Author(s) 2026

Abstract

Smart cities have brought numerous benefits to improve citizens' living quality, but they have also brought greater cyber security challenges. Risk assessment is a commonly used approach to enhance the cyber security of information systems. However, information systems in smart cities are usually large-scale and highly dynamic intelligent network information systems where traditional risk assessment approaches are not applicable. To address this problem, this paper applies artificial intelligence (AI) technology to perform risk assessment as AI-based approach can process big security datasets with fast classification speed and has the ability to discover hidden patterns with higher accuracy. We improved the particle swarm optimization (PSO) algorithm by constructing space symmetric particles and dividing the search process of the PSO algorithm into two stages. On this basis, an IPSO-BPNN (Improved Particle Swarm Optimization-Back Propagation Neural Network) algorithm for security risk assessment of intelligent network information systems is proposed. Following ISO/IEC 27005 and ISO/IEC TS 5689 standards, a risk assessment criterion and risk factors for intelligent network information systems are presented. Also, the overall process of the proposed risk assessment method and the calculation method of risk characteristics are presented. We evaluated the proposed method by comparing it with existing novel AI-based cybersecurity risk assessment methods. Experimental results show that the proposed method can accurately assess cyber security risks in intelligent network information systems with high efficiency.

Keywords Risk assessment · Neural network · Information system · Smart city · Particle swarm optimization · Artificial intelligence

1 Introduction

Urbanization has significantly improved citizens' living quality, but it has also brought challenges such as exponentially increased energy consumption, overburdened healthcare systems and notoriously congested traffic [1]. In this context, smart city technology has become a key of urbanization. The goal of smart city is to utilize information and communications technology to solve urbanization problems. By

applying intelligent information techniques to collect, analyze and process various types of data from the city's routine operation, systems in smart cities can provide citizens with better living environments [1–3]. In this paper, these intelligent information collection and processing systems and service-providing systems that are connected to the internet in smart cities are referred to as intelligent network information systems. Formally, an intelligent network information system can be defined as a large-scale cyber-physical infrastructure that integrates heterogeneous sensing devices, distributed computing nodes, mobile communication networks, and intelligent control units to enable automated data collection, real-time processing, and context-aware service provisioning in urban environments.

In smart cities, traditional information systems and network infrastructure are gradually becoming intelligent [1]. At the same time, traditional information systems and network structures are also constantly expanding. As shown in Fig. 1 (a), a typical traditional information system architecture

✉ Tong Ye
yetong@nuaa.edu.cn

Gongzhe Qiao
qgz@nuaa.edu.cn

¹ College of Computer and Software, Nanjing University of Industry Technology, Yangshan Road, Nanjing 210046, China

² School of Computer Science, School of Cyber Science and Engineering, Nanjing University of Information Science and Technology, Ningliu Road, Nanjing 210044, China

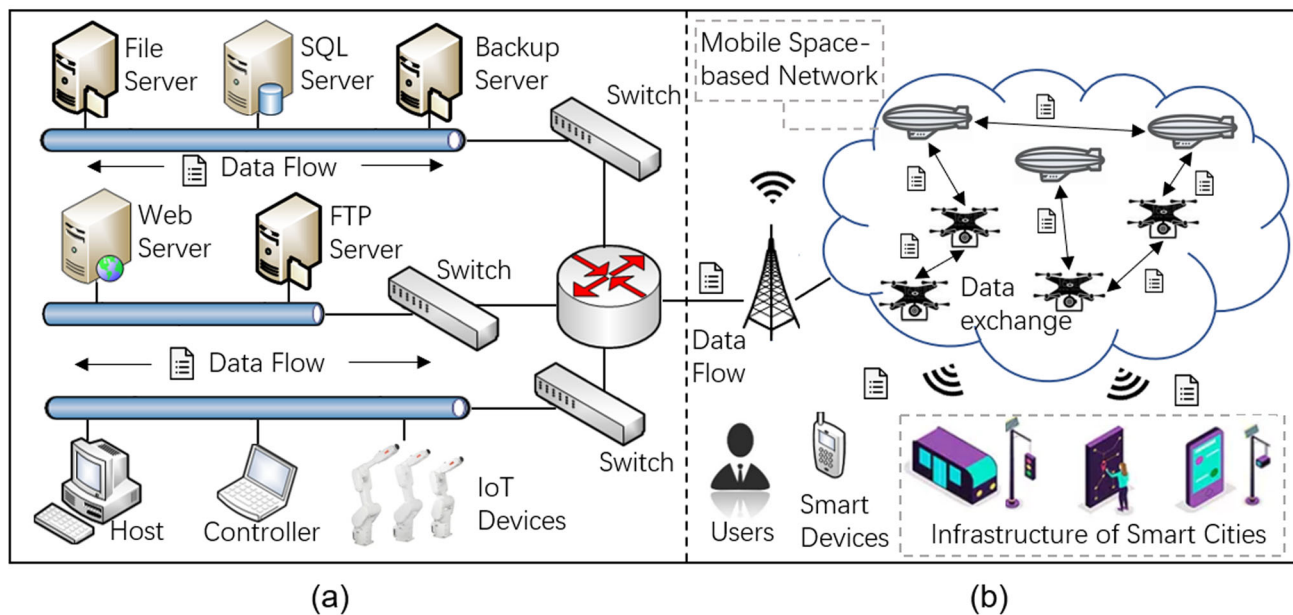


Fig. 1 **a** Traditional information system. **b** The expansion in smart cities

includes switches, servers, hosts, and controllers, where data flows along fixed wired links between centralized servers and terminal devices. This static, wired infrastructure relies on stable physical connections to ensure data transmission, but lacks the flexibility to support dynamic, large-scale mobile scenarios. In the context of smart cities, as shown in Fig. 1 (b), a typical intelligent network information system includes not only city infrastructure but also smart devices controlled by users. Mobile space-based networks such as drone networks are also used, which makes information more widely collected and disseminated. Specifically, this architecture establishes an integrated air-ground-space communication system, consisting of aerial nodes (e.g., unmanned aerial vehicles, airships), terrestrial smart devices, and smart city infrastructure. Data is exchanged dynamically among mobile nodes and transmitted to the ground via wireless access points, enabling on-demand, low-latency data transmission across heterogeneous networks. This decentralized, mobile design addresses the limitations of traditional systems, adapting to the dynamic topology of smart city scenarios to support large-scale mobile communication.

While intelligent network information systems bring convenience to citizens, the hierarchy of the system is gradually increasing, and the architecture and functions are becoming increasingly complex. Therefore, intelligent network information systems are facing increasing security risks. In addition, attackers can steal valuable data from the system and exploit vulnerabilities to attack the system, resulting in a significant decrease in network performance and service quality, and even causing the entire system to be compromised. At the same time, the addition of mobile space-based

networks not only improves network coverage but also brings uncertainty to the reliability of the network due to the dynamic changes in its topology. Intelligent network information systems also face security threats from various aspects such as physical, network, and management levels. Therefore, how to accurately identify system risk factors and evaluate system risks has become a research topic that cannot be ignored.

Risk assessment can serve as a situation awareness approach for systems, enabling staff to promptly identify risks and security threats faced by systems. The existing security risk assessment approaches can be divided into three categories: qualitative, quantitative, and mixed approaches [4]. Among them, commonly used qualitative risk assessment approaches include expert assessment, rating estimates, and checklists of risk sources [5]. Common quantitative risk assessment methods include analytical methods [6] (e.g. sensitivity analysis and scenario analysis), probabilistic theoretical models [7] (e.g. Monte Carlo method and historical simulation method), scenario simulation based on game theory [8], tree constructing methods [9] (e.g. event trees and fault trees), and artificial intelligence (AI) based risk assessment approaches [4] (e.g. neural networks, support vector machines, k-means).

However, intelligent network information systems are generally large-scale and highly dynamic systems consisting of a large number of devices where massive data are collected, transmitted and processed. Using traditional risk assessment approaches, such as expert assessment and checklists of risk sources, to perform accurate and comprehensive assessment of intelligent network information systems is usually

time-consuming and labor-intensive [4]. The AI techniques provide an opportunity to address this problem. AI-based risk assessment approaches can process big security datasets with fast classification speed and have the ability to discover hidden patterns with higher accuracy due to deeper insights and visibility of cybersecurity risks [1]. The neural networks such as back propagation neural network (BPNN) and its optimization methods have already been applied to cyber security risk assessment. However, these methods also have some disadvantages such as slow convergence speed and the problem of local minima. Therefore, how to provide systematic and accurate risk assessment results under limited conditions has become a hot research topic that cannot be ignored. The contributions of this paper are as follows:

- By constructing space symmetric particles, we present an IPSO-BPNN algorithm for risk assessment of intelligent network information systems. Also, we divided the search process of particle swarm optimization in the algorithm into two stages and provided methods for adjusting the inertial coefficient separately.
- Following ISO/IEC 27005 and ISO/IEC TS 5689 standards, we present a risk assessment criterion and risk factors for intelligent network information system. Also, we describe the overall process of the proposed risk assessment method and the calculation method of risk characteristics.
- A simulation environment based on the ONE v1.6.0 simulator is constructed to evaluate the proposed risk assessment method for mobile space-based network communication scenarios within a 500×500 m range. 60 nodes simulating UAVs are randomly deployed to move in uniform linear motion along random directions; random node pairs transmit 50 kB–2 MB packets generated at 1–10 s intervals, with node positions and network communication states recorded every 5 min. Experimental results validate the effectiveness and accuracy of the method, with an average error rate of approximately 1.2%.

The remainder of this paper is organized as follows. The literature is reviewed in Sect. 2. Section 3 describes the improved PSO algorithm and the proposed IPSO-BPNN risk assessment model. In Sect. 4, the process of risk assessment model, risk assessment criterion, risk factors, and the calculation method of risk characteristics are given. Section 5 describes the experimental environment, simulation setup and a comparison of the simulation results obtained from different algorithms. Section 6 concludes the paper.

2 Related work

This section reviews existing security risk assessment methods and examines their inherent limitations in application to highly dynamic and heterogeneous intelligent network information systems.

2.1 Traditional risk assessment methods

Traditional risk assessment methodologies for information systems are broadly categorized into qualitative, quantitative, and hybrid approaches [10]. Qualitative methods including expert evaluation and risk checklists, rely on subjective judgments to categorize risks into ordinal scales [10]. While these approaches offer intuitive interpretation and applicability to early-stage system design, they suffer from significant limitations when applied to smart city environments: assessor bias, inconsistent evaluation standards, and inability to process massive heterogeneous data streams from distributed IoT devices.

Quantitative methods employ probabilistic models and mathematical formalisms to produce numeric risk values [10]. Commonly adopted techniques include Monte Carlo simulation for stochastic risk propagation analysis [11], fault tree analysis (FTA) for system reliability evaluation [12], and scenario-based sensitivity analysis [13]. These methods provide objective, reproducible results but require precise probability distributions and extensive historical failure data—assumptions that rarely hold in dynamic smart city infrastructures where threat landscapes evolve continuously and vulnerability databases remain incomplete.

Hybrid approaches attempt to combine qualitative flexibility with quantitative rigor through fuzzy logic [14], Analytic Hierarchy Process (AHP) [15], and Bayesian networks [16]. However, traditional hybrid methods face scalability constraints: computational complexity increases exponentially with network size, and manual parameter tuning becomes infeasible for large-scale deployments encompassing thousands of heterogeneous devices.

2.2 AI-based risk assessment for smart cities

Traditional risk assessment approaches are not suitable for handling dynamic networks and massive data [1–3]. To address this issue, many research studies have applied AI techniques for security risk assessment in smart cities.

Kalinin et al. [4] analyzed the network security issues faced by smart city network architecture and proposed an AI-based risk assessment approach for smart city networks.

Mouti et al. [17] proposed a network security threat detection and risk management framework based on deep learning. This framework uses probabilistic Bayesian networks to identify attacks and introduces risk values, which can quantitatively calculate cyber security risks. Bitton et al. [18] used the Analytic Hierarchy Process (AHP) to conduct network security risk assessment on intelligent production systems using machine learning. This method can quantitatively calculate the network security risks brought by adversarial machine learning (AML) threats. Rafaiani et al. [19] proposed a model based on machine learning algorithms, which can evaluate security risk levels according to network situations. Hu et al. [20] used hidden Markov models to quantitatively assess cyber security risks. Li et al. [21] proposed an intelligent network security risk assessment method based on decision trees to enhance the stability and security of data communication. Zhao et al. [22] applied big data and machine learning techniques to analyze abnormal behavior and assess security risks in power system networks. Radanliev et al. [23] applied the AutoAI algorithm to assess the cyber security risks of healthcare systems. However, these studies focus on either specific application scenarios or static network environments, lacking adaptability to dynamic mobile space-based networks in smart cities.

Some research studies use metaheuristic algorithms to optimize neural networks to evaluate network situations. Du et al. [24] proposed a network security assessment model based on an improved backpropagation (BP) neural network. They map situation elements to the layers of the neural network and optimize the thresholds and weights of the BPNN through the Cuckoo Search Algorithm (CSA). Zhang et al. [25] optimized the BPNN using Sparrow Search Algorithm (SSA) and Simulated Annealing Algorithm (SAA). They proposed a network security assessment model based on SAA-SSA-BPNN. Gao et al. [26] used the Artificial Bee Colony (ABC) algorithm to optimize neural networks to assess network security risks. Li et al. [27] proposed an evolutionary algorithm based on improved cuckoo search (ICS) and BPNN to improve the accuracy and stability of security risk assessment. These metaheuristic algorithm-optimized models, while improving assessment accuracy to a certain extent, have their own limitations: they often suffer from slow convergence speed, high computational complexity, and are prone to falling into local optimal solutions.

2.3 Risk assessment methods for dynamic environments

While the aforementioned approaches demonstrate efficacy in static or periodically assessed infrastructures, they exhibit limitations when applied to highly dynamic environments such as UAV-assisted mobile networks and smart city CPS with rapidly evolving topologies. Recent stud-

ies have explored adaptive risk assessment frameworks to address these challenges. Czekste et al. [28] presented a lightweight dynamic risk assessment framework utilizing scenario-based discrete-event simulations enriched with synthetic data and threat models for Medical IoT (MIoT) infrastructures. However, the reliance on synthetic data generation and domain-specific medical device models may limit the generalizability of their method to broader CPS environments. Behbehani et al. [29] proposed the Cloud Enterprise Dynamic Risk Assessment (CEDRA) model employing dynamic Bayesian networks augmented with CVSS metrics and real-time threat intelligence feeds to predict vulnerability exploitations and monetize consequential financial losses in cloud computing environments. Nevertheless, CEDRA is primarily designed for centralized cloud infrastructures with relatively static asset portfolios, and its dependency on comprehensive threat intelligence feeds and precise financial metrics may encounter scalability challenges when applied to highly dynamic mobile networks, where real-time asset tracking and continuous connectivity cannot be guaranteed. Additionally, Anwar et al. [30] developed a dynamic risk assessment framework for autonomous vehicle networks. Their framework enables real-time risk score generation for V2V communications in highly mobile vehicular environments. However, the reliance on ruin-theoretic calculations and iterative coalitional game formation introduces considerable computational complexity.

There is an urgent need for an effective risk assessment method that can adapt to the dynamic topology and node mobility characteristics of mobile space-based networks in smart cities, while balancing high accuracy, real-time performance and strong generalization ability. The proposed method in this paper is specifically designed to fill this research gap and overcome the shortcomings of existing studies.

3 Risk assessment model based on improved PSO-BPNN

The target application scenario of this paper is the dynamic intelligent network information system in mobile space-based networks, where dynamic environment characteristics are mainly reflected in real-time changes of network node positions, fluctuations of communication link quality, and time-varying security risk factors (e.g., real-time changes of vulnerability levels, dynamic occurrence of network threats). The improved PSO algorithm and IPSO-BPNN model proposed in this section fully consider these dynamic characteristics, ensuring effective risk assessment in dynamic environments.

3.1 Improved PSO algorithm

The Particle Swarm Optimization (PSO) algorithm uses a swarm approach for searching, which can simultaneously search multiple regions in the solution space of the objective function to be optimized. At the same time, PSO also has the advantages of fast convergence speed, fewer parameters, and easy implementation. The process of the PSO algorithm is as follows.

There are N particles in the D -dimensional particle space. The spatial position of the i th particle is $p_i = (p_{i1}, p_{i2}, \dots, p_{iD})$, with a movement speed of $v_i = (v_{i1}, v_{i2}, \dots, v_{iD})$. To optimize the weights and thresholds of BPNN, we use mean square error to describe the degree of deviation between predicted values and actual sample values. For the training sample set, the fitness function F can be expressed as Eq. (1), where d_j is the expectation of the j th sample, y_j is the actual result of the j th sample, and S is the number of samples.

$$F = \sum_{j=1}^S (d_j - y_j)^2 / S \quad (1)$$

At the end of each round of calculation, each particle will provide the optimal solution F_{pbest} for that round, particle swarm optimization will also provide the optimal population solution F_{gbest} for this round. At the end of each round, particles will compare their own optimal solution with the current group's optimal solution, further updating the group's optimal solution. At the same time, each particle will also update its movement speed and position based on the current individual optimal solution and group optimal solution, and the velocity v_i^{l+1} of the i th particle in the $l+1$ round and position p_i^{l+1} are calculated from equations (2) and (3), respectively; Where Iw is the inertial coefficient, lc_1 and lc_2 are the learning rates, r_1 and r_2 are random numbers between (0,1), v_{ik}^l and p_{ik}^l represents the k -dimensional movement speed and position of the i th particle in the l th round, where k is a positive integer between (1, D).

$$v_i^{l+1} = (Iw \times v_{ik}^l + lc_1 \times r_1 (F_{pbest} - p_{ik}^l) + lc_2 \times r_2 (F_{gbest} - p_{ik}^l)), k \in (1, D) \quad (2)$$

$$p_i^{l+1} = (p_{ik}^l + v_{ik}^{l+1}), k \in (1, D) \quad (3)$$

The particle swarm continuously updates the particle position and velocity to obtain the optimal solution. When the fitness of the particle swarm is less than the given threshold (allowable error), the algorithm is considered to have converged and the program is terminated; Otherwise, the program will terminate when the number of iterations of the algorithm reaches the specified number. Although the particle swarm optimization algorithm has good search ability

and fast convergence speed when solving optimization functions. However, the PSO algorithm is prone to falling into local optima, resulting in significant errors in the results. To address this problem, this paper proposes the concept of symmetric particle pairs and designs an adaptive weight based PSO algorithm.

The PSO algorithm typically initializes N particles randomly, with each particle having D dimensions. As D increases, the range of the objective function solution space increases exponentially. Therefore, the initialized random particle swarm is likely to cluster in a certain range of areas in space, as shown in Fig. 2 (a). This paper divides each dimension into two equal parts (i.e. bisection) and constructs a space balance point. The coordinates of the space balance point are the middle scale of each dimension, as shown in Fig. 2 (b). Each particle has a symmetric particle. A symmetric particle is a mirror symmetric point generated by the particle based on the space balance point, as shown in Fig. 2 (c). The spatial position of the i th symmetric particle is $p'_i = (p'_{i1}, p'_{i2}, \dots, p'_{iD})$ and its movement speed is $v'_i = (v'_{i1}, v'_{i2}, \dots, v'_{iD})$.

In the particle optimization method combined with neural networks, each particle represents a combination of weights between neurons in the neural network. Generally speaking, the weights between neurons can be positive or negative. Therefore, the spatial position of the space balance point in this case can be expressed as $sp = (0, 0, \dots, 0)$. The spatial position relationship between particles and their symmetric particles can be expressed as Eq. (4), and the movement speed relationship between particles and their symmetric particles can be expressed as Eq. (5).

$$p'_i = -p_i, \text{ where } p'_{ik} = -p_{ik} \mid k \in (1, D) \quad (4)$$

$$v'_i = -v_i, \text{ where } v'_{ik} = -v_{ik} \mid k \in (1, D) \quad (5)$$

In the proposed IPSO algorithm, the spatial equilibrium point is fixed at the origin $s_p = (0, 0, \dots, 0)$ of the search space. This setting is established based on the normalized parameter space where all BPNN weights and thresholds are mapped to $[-1, 1]$ through min-max normalization prior to optimization. Consequently, the mirrored particle p'_i calculated by Eq. (4) maintains valid parameter ranges without bias toward positive or negative values, ensuring that the population explores symmetric regions around the center of the normalized solution space.

To address the curse of dimensionality in high-dimensional spaces (where $D = Q(P + 1) + Q + 1$ can be large), we employ dimension-wise selective mirroring: instead of mirroring all D dimensions simultaneously, we randomly select $k = \lceil D/3 \rceil$ dimensions (with $k \geq 2$ to ensure symmetry effects) for mirroring at each iteration. This strategy reduces computational overhead while main-

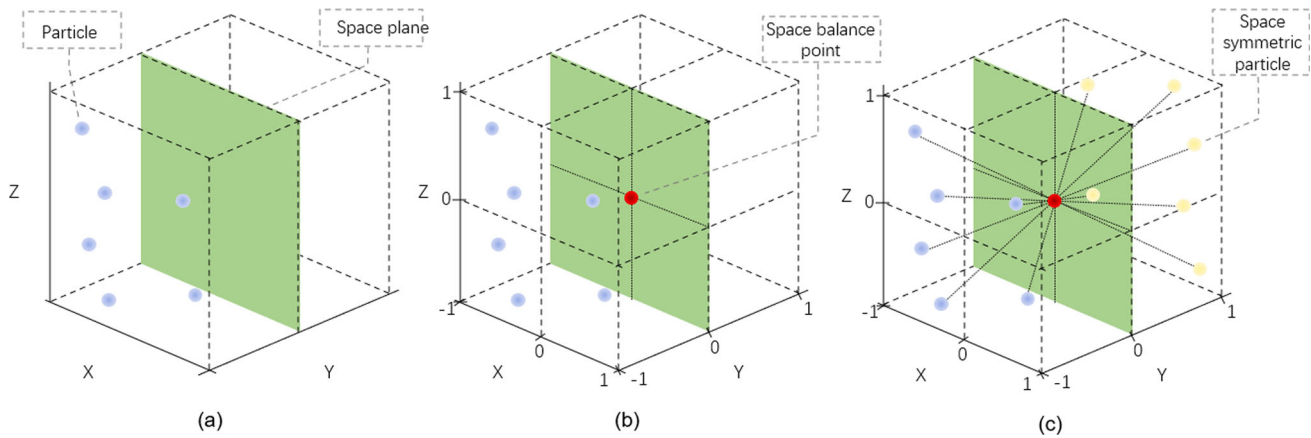


Fig. 2 **a** Randomly initialized particle swarm. **b** The spatial position of the space balance point. **c** Space symmetrical particles

taining population diversity. The dimension selection follows a uniform random distribution without replacement to ensure exploration across different parameter subspaces.

To prevent particle count inflation, the mirrored particles serve as temporary candidate solutions only for fitness evaluation within the current iteration. Following the comparison between p_i and p'_i (Lines 7–13 in Algorithm 1), only the superior particle survives and updates the velocity/position, while the inferior one is discarded. Thus, the population size N remains constant throughout the optimization process. To address spatial expansion issues after mirroring, we implement boundary constraints: if any dimension p'_{ik} exceeds the normalized range $[-1, 1]$, it is reflected back using $p'_{ik} \leftarrow \text{sign}(p'_{ik}) \cdot \min(|p'_{ik}|, 1)$. This reflection mechanism ensures all particles remain within the valid search space without truncation-induced bias, maintaining the integrity of BPNN parameter optimization.

In order to enable the PSO algorithm to find the global optimal solution, this paper divides the search process of particle swarm into two stages: a) particle pair search stage, b) weight linearly decreasing search stage. In the first stage, each particle will compare its own solution with the solution of its symmetric particle and provide the optimal solution for this round. If the optimal solution is obtained from a symmetric particle, the velocity v_i^{l+1} and position p_i^{l+1} of the i th particle in round $l+1$ are calculated by substituting the velocity v'_i and position p'_i of the symmetric particle in round l into equations (2) and (3), respectively. After the first stage of the search, it can be considered that the particle swarm has a certain foundation for large-scale search of the target solution space, which can to some extent avoid falling into local optima. Therefore, in the second stage, particle swarm optimization uses a linearly decreasing weight search method to refine the target solution space and achieve convergence. The weights of the first and second stages (i.e. inertial coefficient Iw) can be expressed as Eq. (6). Among them, $iter$

is the number of iterations in the second stage, which can be expressed as Eq. (7). And $iter_{max}$ is the total number of iterations in the second stage, which can be expressed as Eq. (8). Among them, L is the total number of iterations of the algorithm, and L_f is the total number of iterations in the first stage.

$$Iw = \begin{cases} Iw_{min} + (Iw_{max} - Iw_{min}) \times \text{rand}(0, 1), & \text{in first stage} \\ Iw_{max} - \frac{iter \times (Iw_{max} - Iw_{min})}{iter_{max}}, & \text{in second stage} \end{cases} \quad (6)$$

$$iter = \begin{cases} 0, & \text{if } l \leq L_f \\ l - L_f, & \text{if } l > L_f \end{cases} \quad (7)$$

$$iter_{max} = L - L_f \quad (8)$$

Notably, the adaptive inertial weight (Iw) design of the improved PSO algorithm (Eq. (6)) is an important embodiment of considering dynamic environment characteristics. In dynamic environments, the solution space of the objective function changes in real time with the fluctuation of environmental factors. The two-stage adaptive Iw can adapt to this dynamic change: in the first stage, the random adaptive Iw expands the search range, enabling the algorithm to quickly track the dynamic change of the optimal solution space caused by environmental fluctuations; in the second stage, the linearly decreasing Iw refines the search, ensuring that the algorithm can stably converge to the new optimal solution in the dynamically changing solution space. In addition, the symmetric particle pair design expands the search coverage of the particle swarm, which helps to avoid missing the optimal solution caused by the sudden change of the solution space in dynamic environments, further enhancing the adaptability of the algorithm to dynamic environments.

Algorithm 1 Pseudo Code of IPSO Algorithm**Start**

```

1. FOR each particle  $i$ 
2.   Initialize position  $p_i$  and velocity  $v_i$  randomly within permissible range
3.   Initialize position  $p'_i$  and velocity  $v'_i$  of its space symmetric particle  $i'$ 
4. END FOR
5. Initialize iteration  $l = 1, L, L_f, iter$ 
6. DO
7. IF  $l < L_f$ 
8.   FOR each particle  $i$ 
9.     Calculate fitness values of  $i$  and  $i'$ , and choose a better value
10.    IF the fitness value is better than the  $F_{pbest}$  in history
11.      Set current fitness value as the  $F_{pbest}^l$ 
12.    END IF
13.  END FOR
14. ELSE
15.  FOR each particle  $i$ 
16.    Calculate fitness value
17.    IF the fitness value is better than the  $F_{pbest}$  in history
18.      Set current fitness value as the  $F_{pbest}^l$ 
19.    END IF
20.  END FOR
21. END IF
22. Choose the particle having the best fitness value as  $F_{gbest}^l$ 
23. FOR each particle  $i$ 
24.   Calculate velocity according to equation (2) and (6)
25.   Update particle position according to equation (3)
26. END FOR
27.  $l = l + 1$ 
28. WHILE maximum iterations or minimum error criteria are not attained

```

End

The main idea of this algorithm is to first expand the search range of the objective function solution space using symmetric particle pairs, and then perform a refined particle search. The pseudo-code of the IPSO algorithm is shown in Algorithm 1. The algorithm first initializes the particle swarm and symmetric particles (lines 1-4), and initializes the necessary parameters (line 5). If the number of iterations belongs to the first stage, the optimal solution F_{pbest}^l (lines 7-13) for this round is obtained through the joint search of particles and symmetric particles. If the number of iterations belongs to the second stage, the particle search results in the optimal solution F_{pbest}^l (lines 14-21) for that round. The algorithm selects the optimal solution in the particle swarm as the group's optimal solution F_{gbest}^l for this round (line 22) and calculates the movement speed of each particle in the particle swarm according to Eq. (2) and (6), and updates the position of each particle in the particle swarm according to Eq. (3) (lines 23-26). The algorithm continues to iterate until it satisfies the requirements (lines 27-28).

3.2 The proposed IPSO-BPNN algorithm

The Back Propagation Neural Network (BPNN) has been applied in many fields. Typically, BPNN includes an input layer, a hidden layer, and an output layer. BPNN receives data features through neurons in the input layer and outputs the results obtained by the model through the output layer. This paper mainly uses BPNN to perform cyber security risk assessment for intelligent network information systems. The structure of the neural network used in this paper is as follows.

The first layer of the neural network is the input layer, where P neurons are taken, and the input vector X is the normalized assessment result of the intelligent network information system given by intelligent agents according to the assessment criterion. The second layer is the hidden layer, with Q neurons taken. The third layer is the output layer, where one neuron is taken and the output value is the risk assessment result. We use proposed IPSO to optimize BPNN thresholds and weights. The dimension of a particle is (hidden layer size $\times$ (input layer size + 1)) + (output layer size $\times$ (hidden layer size + 1)), namely $Q \times (P+1) + 1 \times (Q+1)$.

This paper represents the input layer, hidden layer, and output layer as $X = \{x_i \mid i \in (1, P)\}$, $B = \{b_j \mid j \in (1, Q)\}$ and y . Where x_i is the i th neuron in the input layer and b_j is the j th neuron in the hidden layer. The weight value from the i th neuron in the input layer to the h th neuron in the hidden layer is denoted as v_{ih} . The threshold of the h th neuron in the hidden layer is denoted as γ_h . The weight value from the h th neuron in the hidden layer to the output layer is denoted as w_h . The threshold of output layer neurons is denoted as θ . The input of the h th neuron α_h in the hidden layer is the output of all neurons in the input layer. The calculation method for α_h is shown in Eq. (9), and the input of the output layer neuron is β , that is, the output of all neurons in the hidden layer. The calculation method of β is shown in Eq. (10). The output result Y can be calculated by Eq. (11). Among them, f is the activation function of the neural network, and the sigmoid function $f(z) = 1/(1 + e^{-z})$ is selected.

$$\alpha_h = \sum_{i=1}^P v_{ih} \times x_i \quad (9)$$

$$\beta = \sum_{h=1}^Q w_h \times f(\alpha_h - \gamma_h) \quad (10)$$

$$Y = \frac{1}{1 + e^{-(\beta - \theta)}} \quad (11)$$

This paper uses the least squares method to calculate the output error. The output expectation is denoted as op , so that when $i = 0$, $v_{ih} \times x_i = -\gamma_h$. When $h = 0$, $w_h \times f(\alpha_h - \gamma_h) = -\theta$. The calculation method for error E is shown in Eq. (12). To reduce errors, a gradient descent algorithm is used to adjust the weights between neurons in each layer. The gradient calculation method for weights and

thresholds for E is shown in Eq. (13), where WT represents all weights and thresholds in the network, v_{ih} represents the i th neuron in the input layer to the h th neuron in the hidden layer, γ_h is the threshold of the h th neuron in the hidden layer, w_h is the weight value from the h th neuron in the hidden layer to the output layer, and f is the activation function of the neural network. And a particle can be represented as: $(v_{11}, v_{21}, \dots, v_{P1}, \gamma_1, v_{12}, v_{22}, \dots, v_{P2}, \gamma_2, \dots, v_{1Q}, v_{2Q}, \dots, v_{PQ}, \gamma_Q, w_1, w_2, \dots, w_Q, \theta)$.

$$E = \frac{1}{2} \left\{ op - f \left[\sum_{h=0}^Q w_h \times f \left(\sum_{i=0}^P v_{ih} \times x_i \right) \right] \right\}^2 \quad (12)$$

$$\nabla = \frac{\partial E}{\partial WT} \quad (13)$$

BPNN converges slowly under certain conditions and is prone to falling into local minima. As a result, this paper uses IPSO to optimize BPNN. The process of BPNN with IPSO optimization is sketched in Fig. 3. And the steps of this process are as follows.

Step 1: Initialize a BPNN and extract initial thresholds and weights of this neural network for further optimization. Then, the algorithm moves to the IPSO part.

Step 2: Initialize IPSO parameters and each particle represents a combination of thresholds and weights. Then Initialize the position, velocity and space symmetric particle of each particle. Also, the training dataset is input into the model.

Step 3: Following initialization, the algorithm checks the IPSO termination condition. If the current iteration l reaches the maximum L or the global best fitness $F_{gbest} < \varepsilon$ (minimum error threshold, e.g., 10^{-6}), stop the IPSO process and return the best thresholds and weights to BPNN (proceed to Step 4). Otherwise (condition not met), execute the following iterative update process: (i) update individual particle velocity and position using equation (2) and equation (3), respectively; (ii) calculate the fitness value F of each particle using equation (1) based on the training dataset; (iii) update the individual history optimal F_{pbest} and global best F_{gbest} by comparing current fitness with historical values (including elitist selection between particles and their symmetric counterparts as per Algorithm 1, lines 7–13); (iv) update other parameters such as inertia weight using equation (6). After completing the above updates, return to the beginning of Step 3 to re-check the termination condition.

Step 4: Obtain the optimal thresholds and weights corresponding to the global best particle F_{gbest} from the IPSO optimization and return them to the BPNN as the initial parameters.

Step 5: After being optimized by IPSO, the best thresholds and weights are returned to the BPNN and the algorithm back to BPNN part. The BPNN would keep training, calculate error and update thresholds and weights.

Step 6: Check the BPNN training termination condition. If the training epoch reaches the maximum $Epoch_{max}$ (e.g., 100) or the output error E falls below threshold δ (e.g., 10^{-4}), stop the training process and complete the model training. In addition, the prediction results can be obtained through this model.

The IPSO-BPNN algorithm inherits the dynamic adaptability of the improved PSO algorithm, which is the core way to consider dynamic environment characteristics. Since the input vector X of the BPNN (normalized risk factor values) changes in real time with the dynamic environment, the optimal weights and thresholds of the BPNN also need to be dynamically adjusted to ensure assessment accuracy. The IPSO algorithm, with adaptive Iw and symmetric particle pairs, can quickly respond to the real-time changes of the input vector X .

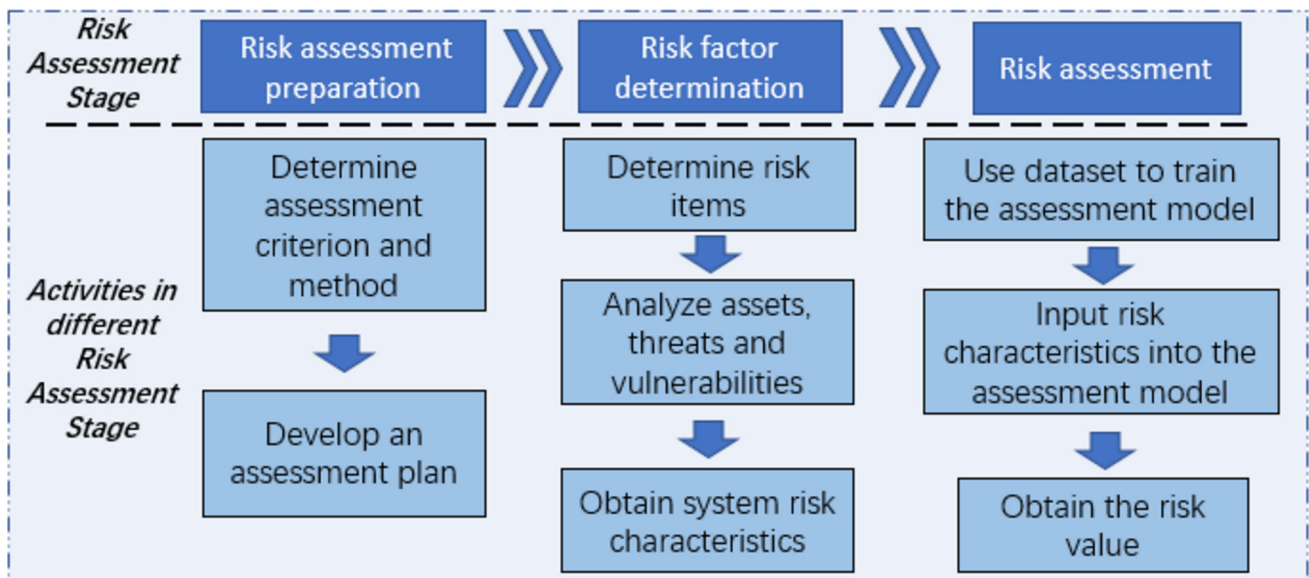
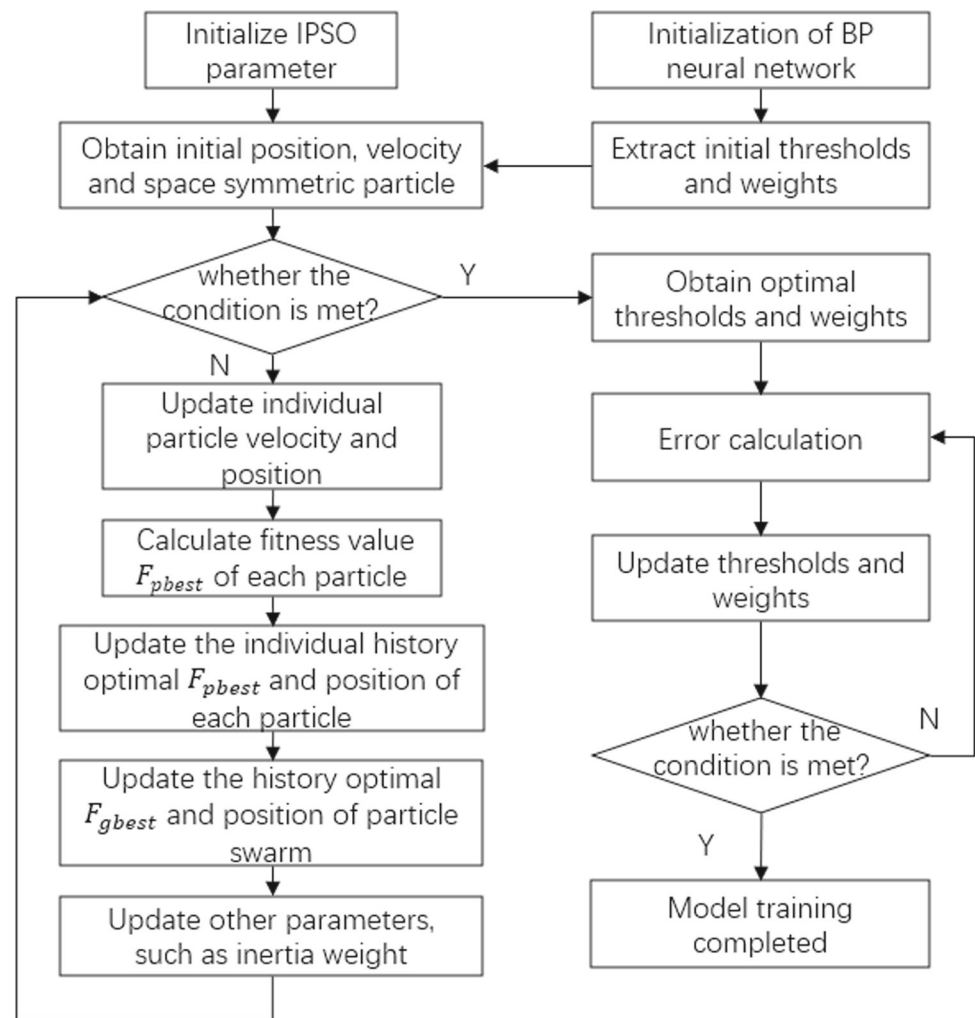
4 Risk assessment framework and assessment criterion

4.1 The process of risk assessment model

Comprehensive security risk assessment of intelligent network information systems is usually time-consuming and labor-intensive. To address this problem, this paper proposes a security risk assessment model based on IPSO-BPNN. This model reduces personnel costs and dependence on experts by introducing artificial intelligence techniques. To make the assessment process conform to the standardized process, we refer to the ISO/IEC 27005 standard and divide the assessment process of the proposed risk assessment model into three stages: a) Risk assessment preparation, b) Risk factor determination and c) Risk assessment. The assessment process is shown in Fig. 4. Each stage can be divided into a series of activities.

Risk assessment preparation: This stage establishes the foundation for subsequent assessment, specifically tailored to intelligent network information systems. The assessment criteria are defined (including the five-level risk classification in Sect. 4.3 and quantitative calculation rules), and the assessment plan is formulated, covering the system scope (e.g., core network nodes, access terminals, and key data modules), evaluation indicators for each link (e.g., asset statistics completeness, vulnerability detection coverage), and schedule and personnel arrangements.

Risk factor determination: This stage identifies system risk factors and analyzes the overall deployment and core functions of the intelligent network information system (e.g., cross-layer data transmission, software stability of intelligent terminals). Specific risk items are determined according to the pre-defined criteria, with detailed analysis of system assets, threats, and vulnerabilities. Meanwhile, unique risk

Fig. 3 The BPNN with IPSO optimization**Fig. 4** The assessment process of the proposed method

characteristics of the intelligent network information system (e.g., high sensitivity of cross-layer threats, strong correlation between software vulnerabilities and data security) are summarized.

Risk assessment: This stage quantitatively evaluates system risk using the proposed IPSO-BPNN model. The dataset is constructed from actual operation data of the intelligent network information system (including historical threat records, vulnerability detection results, and asset value evaluations) to train and optimize the model. The identified risk characteristics (i.e., the 14 risk factors in Section 4.2) are then input into the trained model to output the quantitative risk value.

4.2 Risk assessment criterion and risk factors

Given that the intelligent network information systems targeted in this work operate as cyber-physical systems (CPS), the risk assessment criteria presented herein are designed to align with both ISO/IEC 27005 and ISO/IEC TS 5689 (Cybersecurity – Security frameworks and use cases for cyber physical systems). Taking into account the expansion of the Intelligent network information system on mobile space-based networks, we present a risk assessment criterion including four categories and sixteen risk factors, as shown in Fig. 5. The four categories are management risk, software and data risk, physical equipment risk and network and traffic risk. Also, the risk characteristics can be denoted as $T = \{PM, FM, SV, SC, AC, DS, HF, PE, EU, NA, CT, NI, VP, ST\}$.

Personnel management (PM) and facility management (FM) reference ISO/IEC 27005 organizational controls and address TS 5689 human factors and organizational resilience. Software vulnerability (SV), software configuration (SC), access control (AC), and data security (DS) align with ISO/IEC 27005 technical vulnerabilities and access management, covering TS 5689 system integrity, functional safety, and security-privacy criteria. Hardware failure (HF), physical environment (PE), equipment use (EU), and network architecture (NA) correspond to ISO/IEC 27005 physical threats and network structure and TS 5689 reliability and physical resilience requirements. Communication transmission (CT), network intrusion (NI), virus prevention (VP), and security audit (ST) map to ISO/IEC 27005 transmission protocols and monitoring and TS 5689 security assurance and functional safety. This dual-standard alignment ensures comprehensive coverage of information security risk management and CPS trustworthiness requirements.

The risk value of SV can be evaluated by determining whether there are vulnerabilities and the level of harm caused by them. By checking software configurations such as password strength and remote service connections, the risk value of SC can be determined. The risk value of AC can be

obtained by checking whether the permission allocation and role setting comply with regulations. The risk value of DS can be evaluated by determining whether there is a data leakage pathway or whether the transmitted data is protected (such as encryption).

By detecting whether there are problems with the hardware, whether the environment in which the device is located meets the requirements and whether the use of the equipment complies with the specifications, the risk values of HF , PE and EU can be determined. The risk value of NA is obtained based on indicators such as network connectivity and reliability of communication links.

The risk value of CT can be obtained based on the accuracy of file transmission and the transmission time delay. By examining the response methods for network intrusion, the risk value of NI can be obtained. The risk value of VP can be evaluated by determining whether the virus library is comprehensive and taking preventive measures against the virus. By determining whether to conduct regular security logging and review, the risk value of ST can be evaluated.

4.3 Risk characteristics calculation method

Following the ISO/IEC 27005 standard, we divide the values of system risk and 14 risk factors into five levels: very low, low, medium, high, and very high. The description and value ranges of the five levels are shown in Table 1.

The values of risk factors are calculated based on the assets (A), threats (T), and vulnerabilities (Vul) involved. Referring to Section 8.2.2 and Annex B of ISO/IEC 27005:2018, assets include services, data, software and hardware. Referring to Section 8.2.3 and Annex C of ISO/IEC 27005:2018, threats include software and hardware failures, physical environment impact, inadequate management, malicious code, viruses, and network attacks. Referring to section 8.2.5 and Annex D of ISO/IEC 27005:2018, vulnerabilities include physical environment protection measures, network structure design, software patch, user account, and organization management. A security event (SE) of a risk factor can be represented as Eq. (14).

The 14 risk factors ($PM, FM, SV, SC, AC, DS, HF, PE, EU, NA, CT, NI, VP, ST$) represent distinct categories of security concerns that require assessment. Each risk factor is quantified through underlying security events (SEs), where every SE is fundamentally characterized by three core metrics: Asset value, Threat level, and Vulnerability level. These three metrics thereby provide the quantitative foundation for calculating the risk value of each of the 14 risk factors.

For the asset a_i involved in security incidents SE_i , the asset value val_{a_i} can be given based on the importance of the assets. Based on the frequency of threat t_i occurrence in previous security reports and threat statistics released by international organizations, the threat level val_{t_i} can be eval-

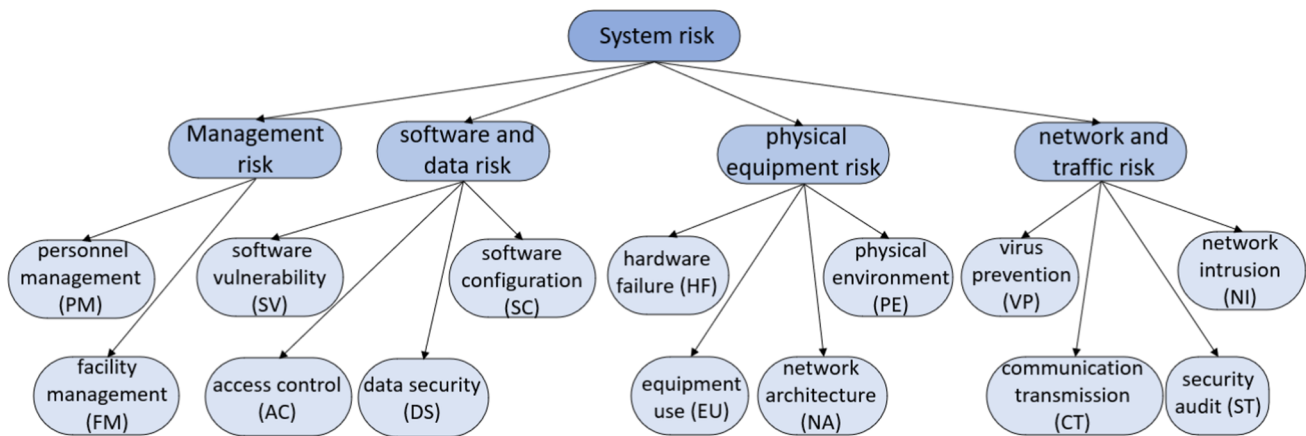


Fig. 5 Risk assessment criterion for intelligent network information system

Table 1 The description and value ranges of the risk levels

Risk Level	Value Range	Description
Very high (5)	[0.8,1]	The risk faced by the system is very high, and it is almost inevitable in most cases. If there are problems, the losses caused will be very serious
High (4)	[0.6,0.8)	The risk faced by the system is high, and it may occur in most cases. If there are problems, the losses caused will be serious
Medium (3)	[0.4,0.6)	The risk faced by the system is medium, and it may occur in some cases. If there are problems, it will cause certain losses
Low (2)	[0.2,0.4)	The risk faced by the system is low, and it is unlikely to occur in some cases. If there are problems, the losses are acceptable
Very low (1)	[0,0.2)	The risk faced by the system is very low, and it is unlikely to occur in most cases. If there are problems, it will cause little damage

uated. Based on the severity of the damage that vul_i may cause to asset a_i , the vulnerability level val_{vul_i} can be determined.

$$SE_i = \langle a_i, t_i, vul_i \rangle \quad (14)$$

Referring to the risk calculation method in Annex E of ISO/IEC 27005:2018 standard, we present the risk value calculation method of risk factors. The occurrence probability of SE_i and the loss caused by SE_i are respectively expressed as PR_i and LS_i , which are shown in Eqs. (15) and (16). The risk value of risk factors (val_{rf}) can be calculated by Eq. (17). Where n is the number of SE of the risk factor.

$$PR_i = \sqrt{val_{a_i} \times val_{vul_i}} \quad (15)$$

$$LS_i = \sqrt{val_{a_i} \times val_{vul_i}} \quad (16)$$

$$val_{rf} = \frac{1}{n} \sum_{i=1}^n \sqrt{PR_i \times LS_i} \quad (17)$$

As shown in Table 2, we use an example to illustrate the calculation method of risk factor value. It is assumed that a risk factor contains three types of security events $\{SE_i \mid i \in (1, 3)\}$. For SE_1 , the asset value of a_1 is val_{a_1} and the relevant vulnerability level of vul_1 is val_{vul_1} . Also, a_1 face the threat t_1 with threat level val_{t_1} . Similarly, we also know the asset values, relevant vulnerability levels and corresponding threat levels in SE_2 and SE_3 . By using formula (15) and (16), we can calculate the occurrence probability (PR_i) of SE_i and the loss (LS_i) caused by SE_i . Then, according to formula (17), we can calculate the risk value of risk factor.

The risk characteristics calculation method proposed in this section further embodies the consideration of dynamic environment characteristics. Since the assets, threats and vulnerabilities in dynamic environments are real-time changing, the calculation of risk factor values (equations 14-17) supports dynamic input of parameters: the asset value (val_{a_i}), threat level (val_{t_i}) and vulnerability level (val_{vul_i}) can be updated in real time according to the current environmental state. For example, when the communication link is unstable (dynamic environmental change), the asset value of the com-

Table 2 Evaluation example of a risk factor

Vulnerability level	Threat level	Asset value	$PR_i \times LS_i$
$val_{vul_1} = 1$	$val_{t_1} = 2$	$val_{a_1} = 2$	$\sqrt{2} \times \sqrt{2} = 2$
$val_{vul_2} = 2$	$val_{t_2} = 3$	$val_{a_2} = 1$	$\sqrt{6} \times \sqrt{2} \approx 3.4641$
$val_{vul_3} = 4$	$val_{t_3} = 1$	$val_{a_3} = 2$	$\sqrt{4} \times \sqrt{8} \approx 5.6568$
Risk Value	$val_{rf} = \frac{\sqrt{2} + \sqrt{3.4641} + \sqrt{5.6568}}{3} \approx 1.8845$		

munication service (val_{a_i}) increases, the threat level of link interruption (val_{t_i}) increases, and the vulnerability level of the link (val_{vul_i}) also changes accordingly; the calculation method can quickly incorporate these dynamic parameter changes, output the real-time risk factor value, and provide accurate input for the IPSO-BPNN model, ensuring that the entire risk assessment process can keep pace with the dynamic environment.

5 Simulations and discussions

5.1 Simulation setup

In order to evaluate the proposed risk assessment approach based on IPSO-BPNN, experimental validation was conducted. The Intelligent network information system in this paper adds intelligent devices and mobile space-based networks to mainstream information systems, as shown in Fig. 1. The assessment object in this study is a smart intelligent network information system, which includes a file server, an SQL server, a backup server, a web server, an FTP server, a smart controller, a host, a firewall, three switches and a router. The main equipment and software in the system are shown in Table 3.

The simulation experiments were conducted using the Opportunistic Network Environment (ONE) simulator v1.6.0, a Java-based platform designed for evaluating routing protocols and applications in mobile wireless networks. ONE integrates movement models, radio propagation, and protocol implementations, enabling realistic performance analysis of opportunistic communication schemes. The scenario size is 500×500 meters, and 60 nodes, which move in a uniform rectilinear motion in a random direction, are randomly distributed to simulate unmanned aerial vehicles (UAVs). The scenario randomly selects nodes to send 50 kB–2 MB sized packets to other nodes and the packets are generated at the rate of one per 1–10 s. Every 5 min, the location of the nodes and the communication status of the network are recorded.

We recorded the status of the system twice a week and performed risk assessment in conjunction with a simulated mobile space-based network. A dataset containing 120 pieces of data was constructed. Among them, 100 pieces of data were used for training the proposed IPSO-BPNN risk assess-

ment model, and 20 pieces of data were used for performance evaluation of the proposed model. The histogram and normal distribution of the data item in the dataset are drawn according to the statistical results, as shown in Fig. 6. The raw monitoring data consist of (1) real-time system logs collected from the equipment listed in Table 2 (including OSSEC-HIDS security events, network device status, and server performance metrics), and (2) simulated mobile network traces generated by ONE v1.6.0 (including UAV node mobility patterns and communication records). These raw data are processed into the 14 risk factor values using the calculation method described in Section 4.3, after which security experts assign an overall risk evaluation value as the label. A piece of data in the dataset consists of 14 risk factor values and an expert evaluation value as a data label. A snippet of the dataset as shown in Fig. 7.

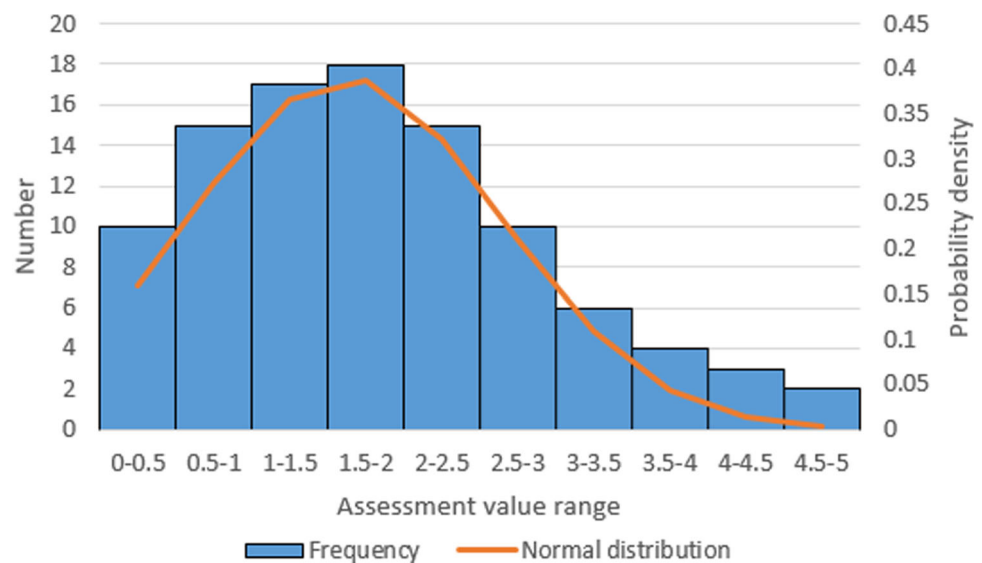
5.2 Evaluation of performance

The performance of the risk assessment method proposed in this paper was measured in terms of accuracy and program runtime. The configuration of the PC for the experiments is: CPU: Intel Core i7-8550, RAM: 8 GB, OS: Ubuntu 18.04, 64-bit. A three-layer structure model is used to build the BPNN. We set 14 neurons at the input layer, 4 neurons at the hidden layer and 1 neuron at the output layer. For the BPNN parts, parameter settings are as follows: epoch = 100 and learning rate = 0.1. For the IPSO parts, we refer to the existing PSO algorithm to set Iw , Ic_1 , Ic_2 . Then we fine tune the parameters during the training process to obtain the parameters with the best performance. Finally, we set $Iw_{max} = 0.8$, $Iw_{min} = 0.2$, $Ic_1 = 1.5$ and $Ic_2 = 1.2$.

Parameter settings are crucial for the performance of the algorithm. we investigated the effects of the population size of the particle swarm and generation of the algorithm on the proposed method as shown in Table 4. Where time means the program runtime and error means the relative error of assessment values. Also, the number of the first stage generation (L_f) is set to half of the total generation. As the generation and population size increase, the accuracy of the algorithm gradually increases. We also found that the program runtime also increases with the increase of these two parameter values. However, when the population size is larger than 100, this parameter has few influences to accuracy. The same trend

Table 3 The main equipment and software in the system

Equipment	Product information	Function and description
File server/SQL server/Backup server/Web server/FTP server	Dell EMC PowerEdge XE8545	It is a host model that provides hardware support for operating systems, software, and services
Server operating system	Ubuntu Server 18.04 LTS MySQL 5.1.60 Apache HTTP	It is an operating system, which is convenient for operators to use Providing database construction, database query, etc Providing HTTP communication, support website deployment, etc
Software	Server V2.4.46 OSSEC-HIDS-2.8.3	Providing system monitoring and control, log monitoring and audit, security event management, etc An automatic intelligent terminal, which provides functions such as distribution network management, fault location, isolation and recovery and asset management
Smart controller	Easergy T300	It is a desktop computer that facilitates users' access to information and services
Host	Dell Optiplex 3050	Providing functions such as threat defense encryption and threat detection
Firewall	Cisco Firepower 4120	Providing functions such as device connection to the network, network packet filtering and forwarding
Switch	Cisco Business 250	Providing centralized management, authentication and access of equipment
Router	TP-LINK AX1800	

Fig. 6 The histogram and normal distribution of the data item in the dataset

(PM)	(FM)	(SV)	(SC)	(AC)	(DS)	(HF)	(PE)	(EU)	(NA)	(CT)	(NI)	(VP)	(ST)	Risk Value
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
0.8678	2.6472	0.3365	0.9414	1.2473	0.2479	0.0041	0.2596	4.0521	0.7286	0.1118	0.6607	0.1293	0.8863	0.9
1.4986	0.23	0.6453	0.8442	0.8497	0.8264	1.1159	1.1674	0.2417	1.8519	0.3392	1.3718	0.1859	1.4169	0.8
0.3179	2.4101	3.4354	3.0576	0.145	2.6693	0.3002	1.9863	2.5143	0.2528	1.0752	0.056	0.1426	0.0036	1.3
0.5525	0.9756	0.3377	2.699	1.079	0.8651	0.387	1.4924	1.1503	0.2463	1.7672	2.2563	3.038	0.1076	1.2
2.4243	0.5652	0.1908	1.2132	0.7872	0.3772	0.6778	3.4885	1.0732	0.3199	0.2926	1.7788	1.39	0.7737	1
0.4545	1.6634	1.2783	0.89	2.3453	0.1813	1.6561	0.5847	1.1477	1.6988	2.4035	0.0213	2.4278	2.669	1.3

Fig. 7 The histogram and normal distribution of the data item in the dataset

Table 4 The effects of population size of particle swarm and generation

Generation	Population									
	20		40		60		80		100	
	Error	Time	Error	Time	Error	Time	Error	Time	Error	Time
20	6.02%	40.56	5.97%	41.96	5.02%	43.67	3.75%	43.95	3.72%	45.02
40	4.25%	41.78	3.98%	43.43	3.92%	45.39	2.52%	45.85	2.43%	47.63
60	3.72%	42.59	3.25%	45.26	2.27%	47.18	1.84%	51.85	1.64%	52.14
80	3.41%	43.21	2.76%	47.59	1.75%	49.89	1.58%	52.79	1.37%	56.27
100	2.27%	44.26	1.97%	48.57	1.54%	53.45	1.32%	57.37	1.21%	59.35

also exists on generation. Therefore, we set the population size and generation to 100 and 100 respectively.

Based on the above parameter settings, we further evaluated the impact of the total number of iterations in the first stage (L_f) on the algorithm performance, as shown in Fig. 8. We evaluated the program runtime and error of the algorithm when the value of L_f was set to 10, 20, 30, 40, 50, 60, 70, 80 and 90, respectively. We found that the program runtime increases with the increase of L_f . This is because the first stage of particle search requires calculation and comparison of double the particle size. Meanwhile, we found that when L_f is set to 30, the algorithm has the optimal accuracy. The reason for this result in our analysis is that the inertial coefficient of the particles in the first stage is relatively large, which is not conducive to the convergence of the algorithm. The inertial coefficient change in the second stage is relatively smooth and smaller than the inertial coefficient value in the first stage. Therefore, in subsequent comparisons with other models, L_f is set to 30 and the error and runtime are 1.18% and 59.32, respectively.

The error rate in Fig. 8 exhibits stochastic fluctuations rather than a smooth declining trend, which reflects the inherent uncertainty in the iterative optimization process. This uncertainty primarily originates from two sources: (i) the random initialization of particle positions and velocities in the first stage, which introduces variability in search trajectories; and (ii) the high inertial coefficient (ω) in early iterations that amplifies the oscillation amplitude of particles, making the convergence path less predictable. To address this uncertainty, we adopt a statistical robustness strategy: each data point in Fig. 8 represents the average of 10 independent runs with different random seeds, and we select $L_f = 30$ not merely based on the single optimal value (1.18%), but considering its consistency across multiple runs (with the lowest variance among all tested values). Furthermore, the second stage with smoother inertial coefficient variation acts as a stabilization mechanism that dampens the uncertainty propagated from the first stage. For future work, we will explore additional strategies to further reduce such uncertainty, including adaptive inertial coefficient tuning in the first stage and dynamic termination criteria based on error

convergence rates, which are expected to enhance the algorithm's robustness against stochastic fluctuations.

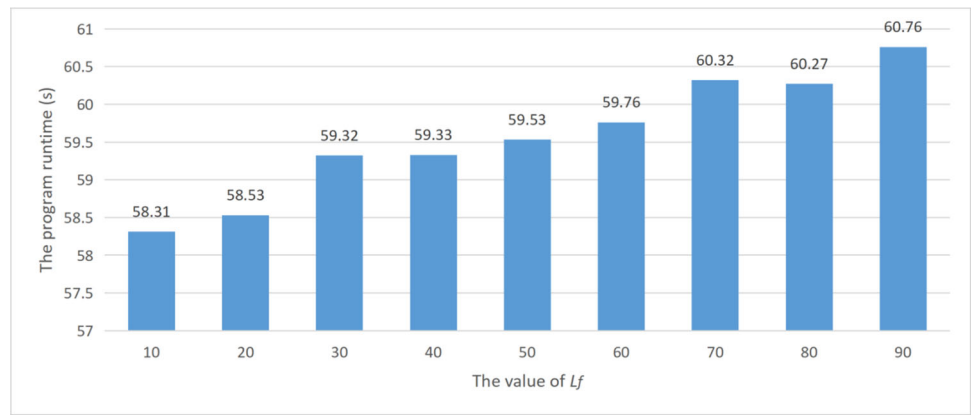
To evaluate the effectiveness of the proposed IPSO algorithm, we select four functions in benchmark [31], which are commonly used for evaluating optimization algorithms. Among them, F1 (Sphere) and F2 (Schwefel) are unimodal functions that can be used to test the convergence speed of optimization algorithms. F9 (Rastrigin) and F10 (Ackley) are multimodal functions that can be used to reflect the ability of optimization algorithms to achieve global optimization. We compared the proposed IPSO algorithm with existing improved particle swarm optimization algorithms [32], [33], [34] and [35]. We conducted 50 experiments on the above five algorithms and calculated the average, standard deviation, and worst values. The experimental results are shown in Table 5. The best results are represented in bold font. It can be seen that the proposed IPSO algorithm has advantages in both unimodal and multimodal functions, and has good global convergence performance.

To further verify the performance of the proposed algorithm, we compared the proposed algorithm with BPNN, PSO-BPNN [36], Cuckoo Search algorithm optimization back-propagation neural network (CS-BPNN) [24] and genetic algorithm optimization back-propagation neural network (GA-BPNN) [37]. For each algorithm, we choose the average results of the ten times as the final results. The error comparison results and the program runtime comparison results are shown in Fig. 9 and Fig. 10, respectively. It can be seen that although there is no obvious advantage in program runtime, the method proposed in this paper has the highest accuracy.

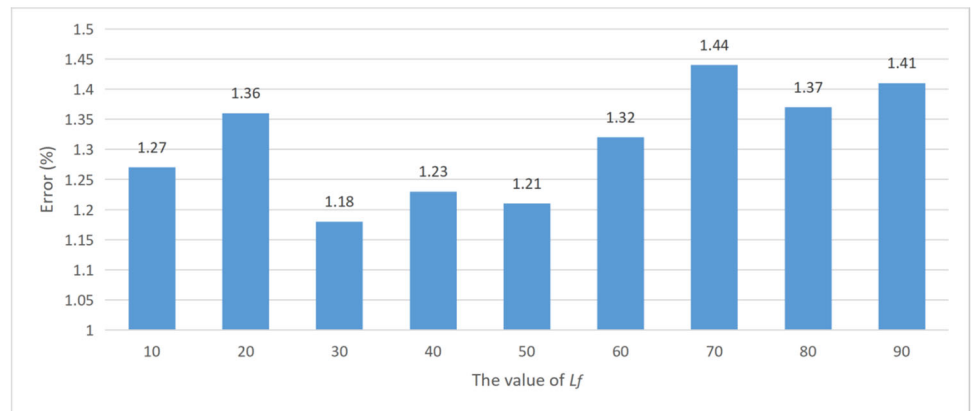
This paper further selected 10 samples to compare the accuracy of PSO-BPNN [36], CS-BPNN [24], GA-BPNN [37] with our method (IPSO-BPNN). The actual value, evaluation value, average error, and average running time of each sample are shown in Table 6. Figure 11 shows the prediction results of different methods for each sample, which can reflect the degree of deviation between the predicted results and the actual values in a more intuitive way.

When comparing the proposed IPSO-BPNN with existing approaches (i.e., standard BPNN, PSO-BPNN [36], CS-BPNN [24], and GA-BPNN [37]), it is important to note

Fig. 8 The impact of the total number of iterations in the first stage on the algorithm performance



(a) The impact of the total number of iterations on the program runtime.



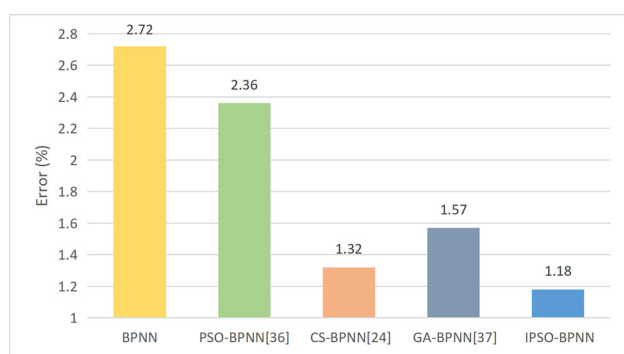
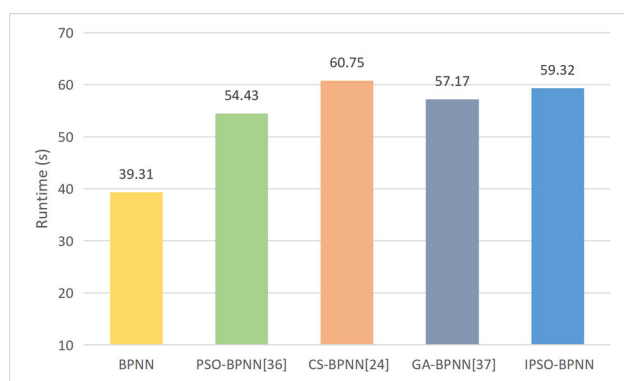
(b) The impact of the total number of iterations on the error rate.

Table 5 The running results of benchmark test functions

Algorithm		IPSO	[32]	[33]	[34]	[35]
F1	Average value	6.1837e−21	2.1562e−13	2.1986e−20	1.0269e−15	1.0879e−24
	Standard deviation	6.6343e−21	1.8936e−13	7.7852e−20	3.0129e−15	1.8235e−24
	Worst value	3.9672e−20	1.2649e−12	3.3492e−19	2.4883e−14	1.0523e−23
F2	Average value	2.4833e−31	4.0428e−19	2.8412e−29	3.9257e−22	2.0612e−22
	Standard deviation	2.2276e−31	2.7478e−18	6.7298e−29	1.0911e−21	5.3854e−22
	Worst value	5.6331e−30	1.7820e−18	3.0940e−28	7.9935e−21	2.1930e−21
F9	Average value	0	1.1128e−27	0	3.0092e−29	2.0512e−31
	Standard deviation	0	1.7323e−27	0	5.7578e−29	2.3075e−31
	Worst value	0	5.2362e−26	0	3.89740e−28	6.0312e−30
F10	Average value	4.2053e−18	1.1053e−9	2.1897e−15	8.5123e−12	1.4196e−17
	Standard deviation	9.0285e−18	3.9739e−9	1.2034e−14	2.3498e−12	2.6108e−17
	Worst value	5.1198e−17	2.0422e−8	5.9093e−14	1.7375e−11	1.5948e−16

Table 6 Actual value, evaluation value, average error, and average running time of each sample

Sample	Actual results	PSO-BPNN	CS-BPNN	GA-BPNN	Our method
1	1.9	1.932	1.887	1.976	1.89
2	2.7	2.573	2.688	2.718	2.677
3	3.2	4.122	3.152	3.225	3.154
4	2.4	1.998	2.428	2.41	2.422
5	3.5	3.454	3.496	3.511	3.466
6	2.2	2.178	2.237	2.291	2.245
7	3.7	3.175	3.659	3.645	3.662
8	2.9	2.218	2.821	2.957	2.931
9	4.1	4.219	4.02	4.126	4.067
10	3.1	3.721	3.033	3.124	3.034
Average error	—	2.53%	1.35%	1.51%	1.17%
Program runtime	—	54.61s	60.74s	57.21s	59.26s

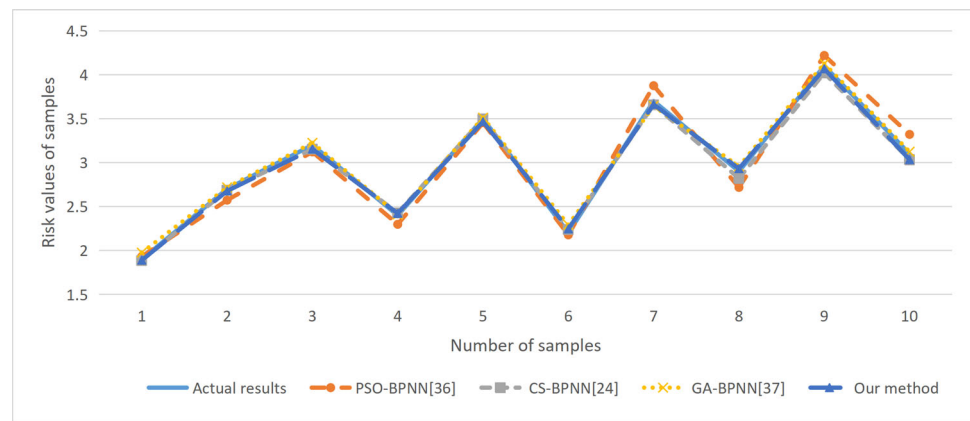
**Fig. 9** Error comparison of different algorithms**Fig. 10** Comparison of the program runtime

that these baseline methods were originally validated on heterogeneous datasets and infrastructural configurations. Such disparities in data provenance, sample scales, and evaluation environments inherently introduce performance biases—particularly in metrics like accuracy and computational overhead. To ensure a fair comparison and isolate the algorithmic contributions from data-specific artifacts, this

work re-implemented all baseline methods within the identical experimental infrastructure. Specifically, all algorithms were evaluated using the same dataset (120 samples with 14 risk factors derived from the ONE-simulated UAV-enhanced CPS environment), identical hardware (Intel Core i7-8550, 8 GB RAM), and uniform BPNN architecture (14-4-1 neurons). Consequently, the observed performance gaps reported in Table 6 and Figures 9–11 reflect intrinsic optimization capabilities rather than variations in training data richness or environmental control.

This paper further selected 10 samples to compare the accuracy of PSO-BPNN [36], CS-BPNN [24], GA-BPNN [37] with our method (IPSO-BPNN). The actual value, evaluation value, average error, and average running time of each sample are shown in Table 6. Figure 11 shows the prediction results of different methods for each sample, which can reflect the degree of deviation between the predicted results and the actual values in a more intuitive way. From the experimental results, we can see that the PSO-BPNN algorithm has the shortest program runtime, and CS-BPNN algorithm has the longest runtime. However, in terms of accuracy, the proposed method (IPSO-BPNN) has the lowest average error. The performance of the proposed method is better than PSO-BPNN and GA-BPNN. And the performance is also slightly better than CS-BPNN. From Fig. 11, it can be seen that the prediction results of PSO-BPNN algorithm are unstable. And the performance of the proposed method is more stable and acceptable. These results indicate that the proposed IPSO-BPNN offers a trade-off between accuracy and stability, making it suitable for risk assessment scenarios where result reliability is prioritized over marginal runtime gains. However, the performance advantage is context-dependent and not universally significant across all metrics.

Fig. 11 Prediction results of different methods for each sample



6 Conclusions

In smart cities, traditional information systems and network infrastructure are gradually becoming intelligent, and network structures are also constantly expanding, which exposes smart cities to more serious cybersecurity risks. To address this challenge, this paper presents an AI-based cybersecurity risk assessment approach for intelligent network information systems. By constructing space-symmetric particles and dividing the search process of the PSO algorithm into two stages, we propose an IPSO-BPNN algorithm for risk assessment. Experimental evaluations against PSO-BPNN, CS-BPNN, and GA-BPNN show that the proposed method yields a comparable prediction accuracy with an error rate of approximately 1.2%, while presenting enhanced stability in prediction performance relative to the comparison methods. The proposed approach has two main limitations. First, the IPSO-BPNN model requires substantial computational resources for particle swarm optimization, which may limit its deployment on resource-constrained UAV edge nodes in real-time applications. Second, the current risk assessment relies on simulated data from the ONE simulator, which may not fully capture the complexity and unpredictability of real-world urban CPS environments with physical device heterogeneity. To address these limitations, future research will focus on: (1) Developing lightweight optimization algorithms (e.g., binary PSO or hybrid metaheuristics) to enable real-time risk assessment on edge devices with limited computing power; and (2) Validating the proposed framework through field experiments in physical smart city testbeds to evaluate its robustness against actual cyber-physical threats and device failures.

Acknowledgements This research was funded by Start-up Fund for New Talented Researchers of Nanjing University of Industry Technology (Grant No. YK24-05-06).

Author Contributions Tong Ye and Gongzhe Qiao wrote the main manuscript text. All authors reviewed the manuscript.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this document.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Panahi Rizi, M.H., Hosseini Seno, S.A.: Systematic review of technologies and solutions to improve security and privacy protection of citizens in the smart city. *Internet of Things* **20**, 100584 (2022)
2. Rao, P.M., Deebak, B.: Security and privacy issues in smart cities/industries: technologies, applications, and challenges. *J. Ambient. Intell. Humaniz. Comput.* **14**, 10517–10553 (2022)
3. Sarker, I.H.: Smart city data science: Towards data-driven smart cities with open research issues. *Internet of Things* **19**, 100528 (2022)
4. Kalinin, M., Krundyshev, V., Zegzhda, P.: Cybersecurity risk assessment in smart city infrastructures. *Machines* **9**(4), 78 (2021)
5. Averyanova, Y., Sushchenko, O., Ostroumov, I., Kuzmenko, N., Zaliskyi, M., Solomentsev, O., Kuznetsov, B., Nikitina, T., Havrylenko, O., Popov, A.: Uas cyber security hazards analysis and approach to qualitative assessment. In: *Data Science and Security: Proceedings of IDSCS 2021*, pp. 258–265. Springer (2021)
6. Kumar, D., Pandey, D., Khan, A.H., Nayyar, H.: Cyber risk analysis of critical information infrastructure: (a quantitative approach). In:

- ISUW 2020: Proceedings of the 6th International Conference and Exhibition on Smart Grids and Smart Cities, pp. 1–9. Springer (2022)
7. Ncubukezi, T.: A proposed: integration of the monte carlo model and the bayes network to propose cyber security risk assessment tool for small and medium enterprises in south africa. *International Journal of Computer Science and Information Security (IJCSIS)* **18**(3), 152–155 (2020)
 8. Amini, M., Bozorgasl, Z.: A game theory method to cyber-threat information sharing in cloud computing technology. *International Journal of Computer Science and Engineering Research* **11**, 4–2023 (2023)
 9. Kong, H.-K., Hong, M.K., Kim, T.-S.: Security risk assessment framework for smart car using the attack tree analysis. *J. Ambient. Intell. Humaniz. Comput.* **9**, 531–551 (2018)
 10. Pan, L., Tomlinson, A.: A systematic review of information security risk assessment. *International Journal of Safety and Security Engineering* **6**(2), 270–281 (2016)
 11. Mondal, S., Singh, R.: Cyber risk propagation and budget optimization in financial networks: a monte carlo approach. *International Journal of System Assurance Engineering and Management*, 1–20 (2025)
 12. Fazlollahab, H., Niaki, S.T.A.: Fault tree analysis for reliability evaluation of an advanced complex manufacturing system. *J. Adv. Manuf. Syst.* **17**(01), 107–118 (2018)
 13. Kim, Y.-G., Cha, S.: Threat scenario-based security risk analysis using use case modeling in information systems. *Security and Communication Networks* **5**(3), 293–300 (2012)
 14. Lee, Z.-J., Chang, L.-Y.: Apply fuzzy decision tree to information security risk assessment. *Int. J. Fuzzy Syst.* **16**(2), 265 (2014)
 15. Eren-Dogu, Z.F., Celikoglu, C.C.: Information security risk assessment: Bayesian prioritization for ahp group decision making. *International Journal of Innovative Computing, Information and Control* **8**(11), 8019–8032 (2012)
 16. Foroughi, F.: Information security risk assessment by using bayesian learning technique. In: *Proceedings of the World Congress on Engineering*, vol. 1, pp. 133–138. (2008). (International Association of Engineers)
 17. Mouti, S., Shukla, S.K., Althubiti, S., Ahmed, M.A., Alenezi, F., Arumugam, M.: Cyber security risk management with attack detection frameworks using multi connect variational auto-encoder with probabilistic bayesian networks. *Comput. Electr. Eng.* **103**, 108308 (2022)
 18. Bitton, R., Maman, N., Singh, I., Momiyama, S., Elovici, Y., Shabtai, A.: Evaluating the cybersecurity risk of real-world, machine learning production systems. *ACM Comput. Surv.* **55**(9), 1–36 (2023)
 19. Rafaiani, G., Battaglioni, M., Compagnoni, S., Senigaglia, L., Chiaraluce, F., Baldi, M.: A machine learning-based method for cyber risk assessment. In: *2023 IEEE 36th International Symposium on Computer-Based Medical Systems (CBMS)*, pp. 263–268. IEEE (2023)
 20. Hu, J., Guo, S., Kuang, X., Meng, F., Hu, D., Shi, Z.: I-hmm-based multidimensional network security risk assessment. *IEEE Access* **8**, 1431–1442 (2019)
 21. Li, Y., Liu, R., Liu, X., Li, H., Sun, Q.: Research on information security risk analysis and prevention technology of network communication based on cloud computing algorithm. In: *Journal of Physics: Conference Series*, vol. 1982, p. 012129 (2021). IOP Publishing
 22. Zhao, Y., Zhang, J., Xiang, S., Tang, Y.: Research on intelligent analysis technology of network security risk based on big data. In: *Journal of Physics: Conference Series*, vol. 1792, p. 012036 (2021). IOP Publishing
 23. Radanliev, P., De Roure, D.: Advancing the cybersecurity of the healthcare system with self-optimising and self-adaptive artificial intelligence (part 2). *Heal. Technol.* **12**(5), 923–929 (2022)
 24. Du, Z., Yao, H., Fu, Y., Cao, Z., Liang, H., Ren, J.: Network situation assessment method based on improved bp neural network. *Electronics* **12**(3), 483 (2023)
 25. Zhang, R., Pan, Z., Yin, Y., Cai, Z.: A model of network security situation assessment based on bpn optimized by saa-ssa. *International Journal of Digital Crime and Forensics (IJDCF)* **14**(2), 1–18 (2022)
 26. Gao, L., Wang, Z., Yang, H., Wang, S., Li, Q., Guo, S., Ma, C.: Network security situation assessment of power information system based on improved artificial bee colony algorithm. In: *Advances in Intelligent Information Hiding and Multimedia Signal Processing: Proceeding of the 16th International Conference on IIHMS in Conjunction with the 13th International Conference on FITAT*, November 5–7, 2020, Ho Chi Minh City, Vietnam, Volume 1, pp. 340–347 (2021). Springer
 27. Li, S., Bi, F., Chen, W., Miao, X., Liu, J., Tang, C.: An improved information security risk assessments method for cyber-physical-social computing and networking. *IEEE Access* **6**, 10311–10319 (2018)
 28. Czekster, R.M., Webber, T., Furstenu, L.B., Marcon, C.: Dynamic risk assessment approach for analysing cyber security events in medical iot networks. *Internet of Things* **29**, 101437 (2025)
 29. Behbehani, D., Komninos, N., Al-Begain, K., Rajarajan, M.: Cloud enterprise dynamic risk assessment (cedra): a dynamic risk assessment using dynamic bayesian networks for cloud environment. *Journal of Cloud Computing* **12**(1), 79 (2023)
 30. Anwar, A., Halabi, T., Zulkernine, M.: A dynamic threat prevention framework for autonomous vehicle networks based on ruin-theoretic security risk assessment. *Journal on Autonomous Transportation Systems* **1**(4), 1–28 (2024)
 31. Mirjalili, S.: SCA: A sine cosine algorithm for solving optimization problems. *Knowl. Based Syst.* **96**, 120–133 (2016)
 32. An, J., He, G., Qin, F., Li, R., Huang, Z.: A new framework of global sensitivity analysis for the chemical kinetic model using PSO-BPNN. *Comput. Chem. Eng.* **112**, 154–164 (2018)
 33. Wang, Z., Wei, J., Rong, H.: An ipso-assisted DNN operation scheme in uav-mounted MEC networks. *Ad Hoc Netw.* **149**, 103258 (2023)
 34. Dhibi, K., Mansouri, M., Trabelsi, M., Abodayeh, K., Bouzrara, K., Nounou, H., Nounou, M.: Enhanced pso-based nn for failures detection in uncertain wind energy systems. *IEEE Access* **11**, 15763–15771 (2023)
 35. Wu, Y., Zhao, D.: Build ipso-abilstm model for network security situation prediction. *J. Inf. Sci. Eng.* **40**(1), 71–88 (2024)
 36. An, J., He, G., Qin, F., Li, R., Huang, Z.: A new framework of global sensitivity analysis for the chemical kinetic model using pso-bpnn. *Computers & Chemical Engineering* **112**, 154–164 (2018)
 37. Li, Y., Xu, X., Wang, W.: Ga-bpnn based hybrid steering control approach for unmanned driving electric vehicle with in-wheel motors. *Complexity* **2018**, 1–15 (2018)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.